

RAG基本框架 - NOTE

其中内容仅为对 RAG 领域内容的概括性总结笔记，仅红色内容为深度探究内容。

后续将随着作者对 RAG 的理解深入，不断对笔记进行丰富。

用户意图理解、知识检索、知识整合、答案生成和RAG评估

1. 用户意图理解 (User Intent Understanding)

- 查询分解(Query Decomposition)

尤其适用于需要多步或组合推理的复杂任务。

- **Least-to-Most Prompting**: 将复杂问题逐步分解为更简单的子问题。--- SCAN 任务
- **Self-Ask**: 让模型自行提出并回答后续问题 --- 多跳问答
- **Chain-of-Verification, CoVe**: 通过独立的事实核查机制，让模型验证自身的回答 --- 列表型问题和长文本生成任务
- **Search-in-the-Chain, SearChain**: 将信息检索 (IR) 集成到推理过程中 --- 多跳问答、事实核查等知识密集型任务

- 查询重写(Query Rewriting)

在解决语义鸿沟 (semantic gaps) 和优化任务结果方面发挥了重要作用。

- **Rewrite-Retrieve-Read, RRR**: 通过 LLM 生成和优化查询，在检索前进行强化学习训练 --
– 开放领域问答 (open-domain QA) 和多项选择任务
- **BEQUE**: 利用有监督微调、离线反馈和对比学习 (contrastive learning) 来弥合语义鸿沟 --
– 电子商务搜索中的长尾查询 (long-tail queries)
- **HyDE**: 零样本 (zero-shot) 方法进行密集检索 (dense retrieval)
- **Step-Back Prompting**: 引导 LLM 从具体示例中抽象出高层概念 --- STEM、复杂推理和知识问答任务

2. 知识检索 (Retrieval)

2.1. 知识来源解析

2.1.1. 结构化知识 ---- KG

- GRAG: 跨文档检索文本子图, 提高检索效率
- KG-RAG: 结合“探索链(CoE)”算法, 增强知识问答能力
- **KG-Retriever**: 分层知识检索器, 由知识图谱层和协作文档层组成的分层索引图上构建索引
- **GNN-RAG**: 采用图神经网络(GNNs)优化KGs 信息处理和推理
- SURGE :结合 KGs 生成知识驱动对话, 提升交互质量

2.1.2. 非结构化知识 ---- PDF

- OCR (光学字符识别) : 解析文本
- PDF 转 Markdown
- 布局分析: 理解页面结构
- 高级解析: 提取表格、公式等

2.1.3. 多模态知识 ---- 图像、音频、视频等

2.2. Embedding

2.2.1. chunk partitioning

- 命题级别划分 (proposition-level) : 按事实单元拆分, 保留关键信息。
- 元块划分 (meta-chunking) : 如 边际采样块划分、困惑度块划分, 增强语义完整性。
- 后期块划分 (late chunking) : 先嵌入全文, 再划分, 适用于复杂文本。

2.2.2. Embedding Models

- Text
 - 静态词嵌入 (word2vec、GloVe、fastText) : 基于上下文编码语义, 但无法动态调整。
 - Transformer 词嵌入(BERT、ROBERTa、ALBERT、DPR): 增强语境理解, DPR被 RAG 采用以提升检索效果。

- 大规模 LLM 嵌入 (BGE、NV-Embed、SFR-Embedding) : 支持多语言和特定领域检索。
- Multimodal

2.3. Knowledge Indexing

- 列表:
 - Table RAG: 文本倒排索引 (inverted index)
- 向量索引 (vector indexing) :
 - 检索:
 - **naive RAG**、ANCE、G-retriever
 - 存储:
 - Chroma 采用分布式存储,
 - Faiss 提供高效相似性搜索,
 - Weaviate 结合图结构优化存储。
- 图索引 (Graph index) :
 - 检索
 - ToG 使用知识图谱
 - **GraphRAG** 提取实体关系并构建文本摘要
 - **RAPTOR** 通过递归嵌入和聚类增强长文本理解
 - 存储:
 - JanusGraph
 - HugeGraph (Gremlin 查询) ,
 - **Neo4j** (Cypher 查询)

2.4. Knowledge Retrieval

	Type	Models	Deep Learning
Retrieval Strategies	Sparse Retrieval	BM25[204]	×
		TF-IDF[205]	×
		likelihood [120]	×
		uniCOIL [139]	✓
		TILDE[290]	✓
		SPLADE[68]	✓
	Dense Retrieval	DPR [112]	✓
		ANCE [252]	✓
		Llama2vec [129]	✓
		RepLLaMA [159]	✓
	Hybrid Retrieval	RAP-Gen[238]	✓
		BlendedRAG[213]	✓
		ReACC[151]	✓
		BASHEXPLAINER[265]	✓
Search Approaches	Nearest Neighbor Search	Brute Force Algorithm	×
		KD Tree [19]	×
		Ball Tree [54]	×
		M-tree[37]	×
	Approximate Nearest Neighbor Search	Locality-Sensitive Hashing[6, 45]	×
		Spectral Hashing[242]	×
		Deep Hashing[143]	✓
		K-means tree[228]	×
		ANNOY	-
		HNSW [161]	×
		Product Quantization[99]	×

Table 3. Knowledge Retrieval Models

2.4.1. 检索策略 → top-k chunks

- 稀疏检索(Sparse Retrieval): 基于关键词匹配, 使用 BM25、TF-IDF 等计算相关性 --- 速度快但难以理解语义。
- 稠密检索(Dense Retrieval): 基于深度学习, 如 DPR、ANCE, 将文本转为向量, 计算余弦相似度 --- 计算开销较大。
- 混合检索(Hybrid Retrieval): 如 BlendedRAG

2.4.2. 搜索方法

- NNS (最近邻搜索): 遍历所有向量, 计算与查询的距离 --- 最精确但计算量大
 - k-d 树
 - R 树
 - M 树
- ANNS (近似最近邻搜索): 在搜索速度和精度间折中 --- 适用于大规模数据
 - 哈希方法 (如 LSH、深度哈希): 将高维向量映射到低维哈希空间, 加速搜索。
 - 树结构方法 (如 K-means 树、ANNOY): 层次化组织数据, 减少搜索范围。
 - 图结构方法 (如 HNSW): 构建数据点之间的邻接关系, 快速遍历找到相似向量。

- 量化方法（如 PQ）：通过数据压缩减少存储，提高查询效率。

3. 知识融合 (Knowledge Integration | Augmentation)

- 输入层融合(Input-Layer Integration): -> 检索信息与原始查询融合
 - **文本级整合**: 直接将前 k 个检索文档与查询拼接, 并通过**重排序**、**加权过滤**、**上下文压缩**等方法优化输入, 减少冗余信息。
 - **特征级整合**: 将输入和检索文档转换为**特征表示** (如稠密/稀疏向量), 提高信息融合的灵活性和效率。
- 中间层融合(Intermediate-Layer): -> 整合至模型的隐藏层
 - **基于注意力机制**:
 - RETRO 模型使用交叉注意力融合检索信息,
 - TOME 采用 MentionMemory 存储和检索实体信息,
 - LongMem 结合残差网络和注意力机制提升长期记忆检索能力。
 - **基于加权加法**: 为检索到的文档学习权重, 通过加权嵌入相加的方式整合信息。
- 输出层融合(Output-Layer Integration): -> 在模型的输出层融合检索知识
 - **基于集成的方法**: 如 KNN-LM 通过**插值检索概率与模型预测概率**提高泛化能力。
 - **基于校准的方法**:
 - EDITSUM 采用**原型 logits**优化代码摘要生成
 - MA 用于**图像描述**任务。答案生成 (Generation)

4. 答案生成 (Answer Generation)

- **去噪(Denoising)**: 侧重于从检索到的知识中筛除无关或相互矛盾的信息, 确保仅有可靠且相关的知识影响生成过程。
 - **显式去噪**: 如 InstructRAG 生成推理步骤过滤噪声, REFEED 通过 LLM 重新评估答案准确性
 - **基于鉴别器去噪**: 如 COMBO 使用预训练鉴别器评估生成段落的连贯性和相关性, 减少幻觉。
 - **自反思与自适应去噪**: Self-RAG 让模型自我评估并迭代优化, 动态筛除低质量信息。

- **上下文过滤与置信度评分**: 模型根据匹配度评分过滤低置信度文档，尤其适用于开放域问答任务。
- **推理(Reasoning)**：则使模型能够有效整合来自多个来源的信息，建立逻辑联系，并生成有据可依的输出。
 - **基于结构化知识的推理**: 如 Think-on-Graph 2.0 将知识图谱与文本结合，提升复杂关系推理能力。
 - **跨注意力机制推理**: 如 RETRO 采用块级跨注意力聚合多文档信息，kNN 注意力层增强上下文一致性。
 - **基于记忆的推理**: 如 TOME 维护实体特定的记忆模块，确保长期一致性和多步推理能力。
 - **检索校准与选择性推理**: 校准技术帮助模型优先选择关键检索信息，避免无关内容干扰推理。
 - **层次化和多轮推理**: 通过逐步优化信息理解适用于因果关系和时间依赖性强的任务

4.1. 知识引用 (Knowledge Citation)

确保 RAG 模型的透明度、可信度和事实依据，减少幻觉现象。

引用方法：

- **同步引用生成**：在生成答案的同时检索信息，确保答案和引用对齐（如 WebGPT、GopherCite）
- **后生成引用检索**：先生成答案，再搜索引用（如 RARR、LaMDA） --- 计算效率高但可能引用不一致

引用粒度：

- **粗粒度引用 (coarse-grained references)**：引用整个文档或 URL（早期模型如 LaMDA）
- **细粒度引用 (fine-grained citations)**：（如 WebGPT、WebBrain 和 GopherCite）
- **句级引用**：
 - longcite 提出的 CoF (Coarse to Fine) 管道利用大语言模型 (LLM) 自动生成精准的句级引用；
 - Think&Cite 则将句子生成与归因建模为多跳推理(multi-hop reasoning)问题，并使用自引导蒙特卡洛树搜索(Self-Guided Monte Carlo tree search)求解。

4.2. RAG 训练

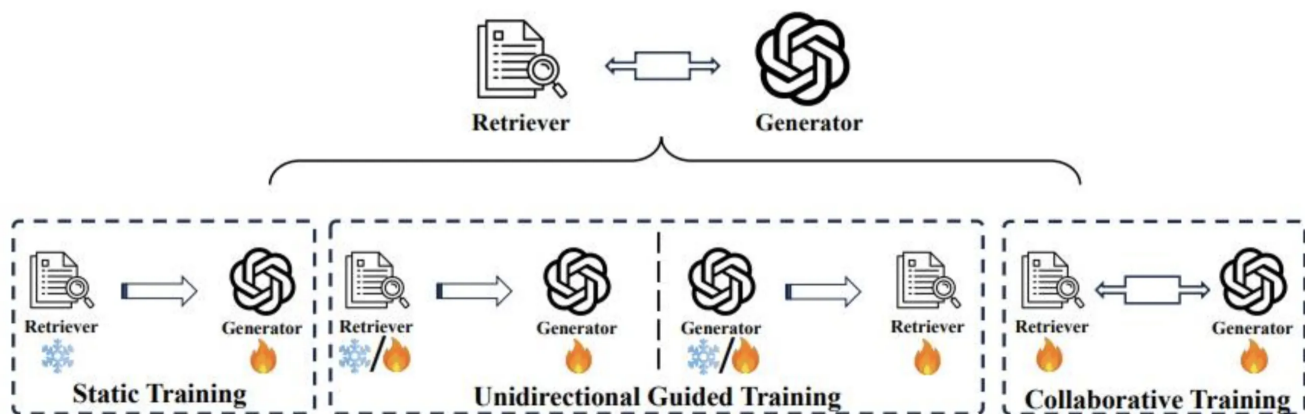


Fig. 7. RAG training includes static training, unidirectional guided training, and collaborative training.

知乎 @塞大花

- 静态训练
- 单向引导训练
- 协同训练

5. RAG评估

5.1. 有效性

- 查询-上下文相关性
 - ARES、RAGAS、TruLens: 基于 上下文相关性 (Context Relevance) 衡量匹配度
 - KILT: 通过 R-precision、Recall@k 评估检索内容的完整性
- 上下文-答案连贯性
 - CRUD: 用 召回率、精准率 评估答案正确性
 - ARES、RAGAS: 采用 答案忠实度 (Answer Faithfulness), 确保内容无幻觉
 - TruLens: 引入 信息扎根性 (Groundedness), 逐句检查答案与上下文的匹配度
 - LongBench-Cite、RECLAIM: 关注 高质量引用和归因, 确保透明可信
- 查询-答案准确性
 - CRAG: 计算 幻觉率、遗漏率, 确保事实正确性
 - RAGAS、ARES、TruLens: 采用 答案相关性 (Answer Relevance) 评估匹配度
 - KILT、RAGQuestEval: 面向任务优化答案质量分析
 - RGB 框架: 扩展评估指标, 如 错误检测率、修正率、拒绝率, 强化错误控制能力

5.2. 效率

- 延迟 (Latency) : 端到端响应时间, 受 **检索和生成方式** 影响
- 吞吐量 (Throughput) : 每秒可处理的查询量, 反映系统扩展能力
- 资源利用率 (Resource Utilization) : CPU/GPU 使用率、内存消耗, 衡量硬件效率

优化策略:

- 稠密检索 (Dense Retrievers) 提高准确率, 但计算开销较高
- RAG-Performance 框架可模拟高负载环境, 测试不同配置下的系统性能